

tailer.py Deep Dive

What This Script Is Responsible For

`src/manasight_parser_py/log/tailer.py` is the file-watching layer for the parser.

Its job is very narrow:

- watch MTGA's `Player.log`
- remember how far into the file we have already read
- read only the newly appended bytes
- turn those new bytes into `LogEntry` batches with help from `entry.py`
- notice when the log file gets reset or rotated

This script does **not** understand Magic gameplay yet.

That happens later in the pipeline:

- 1 `tailer.py` reads new text from the file
- 2 `entry.py` groups that text into log entries
- 3 `router.py` decides which parser should handle each entry
- 4 `parsers/*` turn entries into typed MTGA events
- 5 `state.py` and `gameplay_actions.py` build higher-level match and game meaning

The Main Types In This File

`TailerError`

This is a custom exception.

A custom exception is a project-specific error type. It makes failures easier to understand and easier to catch in higher layers like `stream.py`.

`TailerError` is used when:

- the log file does not exist at startup
- the log file disappears while the parser is running

Its messages now use a sanitized label like `Player.log` instead of the full local Windows path.

`TailBatch`

This is a small return object with two fields:

- `entries`: the parsed `LogEntry` objects from the newest chunk of file data
- `rotated`: whether the log file appears to have been reset or replaced

This is helpful because the caller needs both pieces of information at once:

- the actual content
- the lifecycle signal that the file changed shape

FileTailer

This is the main class.

It stores the file path and the tailing state:

- `_offset`: how many bytes we have already consumed
- `_buffer`: a `LineBuffer` from `entry.py` for partial multi-line entries
- `_seconds_without_structured_headers`: a health counter used by `stream.py`
- `_poll_interval_seconds`: how long one poll waits before checking again
- `_last_poll_monotonic`: the last monotonic clock reading used to measure real elapsed time

How The Script Works Now

Startup Modes

There are two class methods for creating a tailer:

`open_from_start(...)`

This starts reading from byte 0.

Use this when you want to replay the entire existing file.

`open_from_end(...)`

This starts reading from the current file size.

Use this when you only care about new live data and want to ignore old log history.

The parser currently uses `open_from_end(...)` in `stream.py` so startup does not replay stale matches from older MTGA sessions.

What `poll_once()` Does

`poll_once()` is now the real "do one file check" method.

Each call does this:

- 1 measure real elapsed time since the previous check
- 2 offload the blocking file check into `asyncio.to_thread(...)`
- 3 inside that worker thread, inspect the file size and detect whether it shrank
- 4 if it shrank, treat that as rotation and reset the read offset
- 5 if no new bytes were added, return an empty batch
- 6 otherwise read only the appended bytes
- 7 decode those bytes with `errors="replace"` so malformed bytes remain visible

- 8 send the decoded text into `LineBuffer`
- 9 if the chunk ends with a newline, flush the buffered entry
- 10 record whether this batch included structured MTGA headers
- 11 update the "seconds without structured headers" health signal using real elapsed time
- 12 return a `TailBatch`

What `poll()` Does

`poll()` is now a compatibility wrapper.

It does only two things:

- 1 sleep for the configured poll interval
- 2 call `poll_once()`

Why The Recent Cleanup Was Worth Doing

Before the cleanup, `poll()` worked, but it had three quality problems:

- 1 it reread the entire log file every poll
- 2 it kept a mostly dead `_last_size` field
- 3 almost all behavior lived in one method

That meant the script was:

- less efficient than it needed to be
- harder to read
- harder to test in focused ways

What Changed In The Cleanup

The updated version keeps the same job and same role in the pipeline, but it is cleaner internally.

1. It now reads only appended bytes

Instead of:

- reading the whole file into memory
- slicing off the unread tail

it now:

- opens the file in binary mode
- seeks to the saved `_offset`
- reads from there to the end

This is a better fit for a growing log file.

1b. The blocking file work is now offloaded from the event loop

The code still uses normal local file APIs, but it now runs that blocking work inside `asyncio.to_thread(...)`.

That means:

- disk I/O is moved off the main async event loop
- `poll_once()` now has a real reason to stay async
- the parser loop is less likely to pause while waiting on the filesystem

2. The dead `_last_size` field was removed

That field was being written but not meaningfully used.

Removing dead state is a quality improvement because it lowers confusion for future maintenance.

3. `poll()` was split into `poll_once()` plus smaller helpers

This was the main structural improvement.

Now there is a clean distinction between:

- "read the file one time"
- and
- "wait before the next read"

That makes timing control more flexible and easier to reason about.

The behavior is now easier to follow because the internal steps are named:

- `_ensure_log_exists(...)`
- `_existing_size(...)`
- `_read_poll_snapshot(...)`
- `_parse_entries(...)`
- `_record_unstructured_poll(...)`
- `_update_structured_header_health(...)`

That makes the file easier to teach, debug, and extend.

4. Poll timing is now configurable, and the stream can own scheduling

The class still has a `poll_interval_seconds` setting.

But `stream.py` now calls `poll_once()` and owns the outer sleep loop.

That is a more professional separation of responsibilities:

- `tailer.py` owns file reading
- `stream.py` owns scheduling

Tests can still set the interval to 0, which makes focused validation much faster.

What Is Correct About This Script

- It has a clear single responsibility.
- It does not mix gameplay logic into file reading.
- It handles log rotation in a simple way.
- It works well with `entry.py` by preserving partial lines between polls.
- It gives `stream.py` a useful health signal through `seconds_without_structured_headers`.
- That signal now reflects real elapsed time instead of raw poll count.
- Decode problems are visible instead of being silently dropped.
- The event loop no longer does raw blocking disk reads directly.

What Is Still Not Ideal

Even after cleanup, there are a few design tradeoffs worth understanding.

1. File I/O is still thread-offloaded, not truly native async

This is much better for the event loop, but it still relies on normal local file APIs under the hood.

2. Rotation detection is size-based

The current logic treats "file size got smaller" as rotation or truncation.

That is a reasonable heuristic, but it is still a heuristic. It is simple rather than perfect.

3. Sanitized paths trade precision for privacy

The new error messages no longer include the full local Windows path.

That is better for privacy, but it gives slightly less location detail during debugging.

How This File Fits With `stream.py` And `entry.py`

There are now two important relationships:

- `stream.py` owns the polling loop timing
- `tailer.py` owns the one-shot file read behavior

And:

- `tailer.py` owns file offsets and byte reading
- `entry.py` owns line grouping and header detection

That split is good.

It means:

- `tailer.py` does not need to understand MTGA entry structure deeply
- `entry.py` does not need to care about files, offsets, or truncation

The Most Important Takeaway

If you want one sentence:

`tailer.py` is the "read only the new part of the file safely" module.

It is one of the earliest layers in the project, and it should stay boring, predictable, and fast.

That is exactly why the cleanup focused on:

- smaller helpers
- less dead state
- less unnecessary file reading

instead of changing its purpose.